

Troubleshooting

Troubleshooting with Docker

Generating Debugging Logs

For most common issues, you can generate debugging logs to help identify the root cause of the problem. To generate debug logs:

- Edit your runtime configuration under `/etc/nvidia-container-runtime/config.toml` and uncomment the `debug=...` line.
- Run your container again to reproduce the issue and generate the logs.

Generating Core Dumps

In the event of a critical failure, core dumps can be automatically generated and can help troubleshoot issues. Refer to [core\(5\)](#) to generate these. In particular, check the following items:

- `/proc/sys/kernel/core_pattern` is correctly set and points somewhere with write access.
- `ulimit -c` is set to a sensible default.

If the `nvidia-container-cli` process becomes unresponsive, you can also use [gcore\(1\)](#).

Sharing Your Debugging Information

You can attach a particular output to your issue with a [drag and drop](#) into the comment section.

Conflicting values set for option Signed-By error when running apt update

When following the installation instructions on Ubuntu or Debian-based systems and updating the package repository, you might see the following error:

```
$ sudo apt-get update
E: Conflicting values set for option Signed-By regarding source https://nvidia.github.io/
E: The list of sources could not be read.
```

This is caused by the combination of two things:

1. A recent update to the installation instructions to create a repo list file

```
/etc/apt/sources.list.d/nvidia-container-toolkit.list
```

2. The deprecation of `apt-key` meaning that the `signed-by` directive is included in the repo list file

If this error is triggered, it means that another reference to the same repository exists that does not specify the `signed-by` directive. The most likely candidates would be one or more of the files `libnvidia-container.list`, `nvidia-docker.list`, or `nvidia-container-runtime.list` in the folder `/etc/apt/sources.list.d/`.

The conflicting repository references can be obtained by running and inspecting the output:

```
$ grep "nvidia.github.io" /etc/apt/sources.list.d/*
```

The list of files with possibly conflicting references can be obtained by running:

```
$ grep -l "nvidia.github.io" /etc/apt/sources.list.d/* | grep -vE "/nvidia-container-toolkit"
```

Deleting the listed files should resolve the original error.

Permission denied error when running the nvidia-docker wrapper under SELinux

When running the `nvidia-docker` wrapper (provided by the `nvidia-docker2` package) on SELinux environments, you might see the following error:

```
$ sudo nvidia-docker run --gpus=all --rm nvcr.io/nvidia/cuda:11.6.2-base-ubuntu20.04 nvidia-docker
/bin/nvidia-docker: line 34: /bin/docker: Permission denied
/bin/nvidia-docker: line 34: /bin/docker: Success
```

With SELinux reporting the following error:

```
SELinux is preventing /usr/bin/bash from entrypoint access on the file /usr/bin/docker.
SELinux is preventing /usr/bin/bash from entrypoint access on the file /usr/bin/docker.
```

```
***** Plugin catchall (100. confidence) suggests *****
```

```
If you believe that bash should be allowed entrypoint access on the docker file by default.
Then you should report this as a bug.
```

```
You can generate a local policy module to allow this access.
```

```
Do
```

```
allow this access for now by executing:
```

```
# ausearch -c 'nvidia-docker' --raw | audit2allow -M my-nvidiadocker
# semodule -X 300 -i my-nvidiadocker.pp
```

This occurs because `nvidia-docker` forwards the command line arguments with minor modifications to the `docker` executable.

To address this, specify the NVIDIA runtime in the `docker` command:

```
$ sudo docker run --gpus=all --runtime=nvidia --rm nvcr.io/nvidia/cuda:11.6.2-base-ubuntu
```

Alternatively, you can generate a local SELinux policy as suggested:

```
$ ausearch -c 'nvidia-docker' --raw | audit2allow -M my-nvidiadocker
$ semodule -X 300 -i my-nvidiadocker.pp
```

NVML: Insufficient Permissions and SELinux

Depending on how your Red Hat Enterprise Linux system is configured with SELinux, you might have to specify `--security-opt=label=disable` on the Docker or Podman command line to share parts of the host OS that cannot be relabeled. Without this option, you might observe this error when running GPU containers: `Failed to initialize NVML: Insufficient Permissions`. However, using this option disables SELinux separation in the container, and the container runs in an unconfined type. Review the SELinux policies on your system.

Containers losing access to GPUs with error: “Failed to initialize NVML: Unknown Error”

When using the NVIDIA Container Runtime Hook (that is, the Docker `--gpus` flag or the NVIDIA Container Runtime in `legacy` mode) to inject requested GPUs and driver libraries into a container, the hook makes modifications, including setting up cgroup access, to the container without the low-level runtime (such as `runc`) being aware of these changes. The result is that updates to the container can remove access to the requested GPUs.

When the container loses access to the GPU, you see the following error message from the console output:

```
Failed to initialize NVML: Unknown Error
```

The message can differ depending on the type of application that is running in the container.

You need to delete the container after the issue occurs. When it is restarted, either manually or automatically depending on whether you use a container orchestration platform, it regains access to the GPU.

Affected Environments

On certain systems, this behavior is not limited to *explicit* container updates such as adjusting CPU and memory limits for a container. On systems where `systemd` is used to manage the cgroups of the container, reloading the `systemd` unit files (`systemctl daemon-reload`) is sufficient to trigger container updates and cause a loss of GPU access.

Mitigations and Workarounds

⚠ Warning

Certain `runc` versions show similar behavior with the `systemd` cgroup driver when `/dev/char` symlinks for the required devices are missing on the system. Refer to [GitHub discussion #1133](#) for more details around this issue. The behavior persisted even if device nodes were requested on the command line. Newer `runc` versions do not show this behavior and newer NVIDIA driver versions ensure that the required symlinks are present, reducing the likelihood of the specific issue occurring for affected `runc` versions.

Use the following workarounds to prevent containers from losing access to requested GPUs when a `systemctl daemon-reload` command is run:

- For Docker, use cgroupfs as the cgroup driver for containers. To do this, update the `/etc/docker/daemon.json` to include:

```
{
  "exec-opts": ["native.cgroupdriver=cgroupfs"]
}
```

and restart docker by running `systemctl restart docker`. This ensures that the container does not lose access to devices when `systemctl daemon-reload` is run. This approach does not change the behavior for explicit container updates, and a container still loses access to devices in this case.

- Explicitly request the device nodes associated with the requested GPU(s) and any control device nodes when starting the container. For the Docker CLI, this is done by adding the relevant `--device` flags. In the case of the NVIDIA Kubernetes Device Plugin, the `compatWithCPUManager=true` [Helm option](#) ensures the same result.
- Use the Container Device Interface (CDI) to inject devices into a container. When CDI is used to inject devices into a container, the required device nodes are included in the modifications made to the container config. This means that even if the container is updated, it still has access to the required devices.

“cannot find name for group ID” warning when running containers

When running GPU containers, you might see a warning such as the following from the console output:

```
groups: cannot find name for group ID 993
```

The group ID in the message can differ depending on your system, and the message can differ depending on the type of application that is running in the container.

To allow non-root users in a container to access certain device nodes (for example, `/dev/dri/renderD128`), the NVIDIA Container Toolkit adds the group IDs that own these device nodes on the host as additional group IDs for the container. Because these group IDs are resolved from the host, a matching group entry does not necessarily exist in the container’s `/etc/group` file. When a tool such as `groups` or `id` tries to resolve the name for such a group ID and does not find a matching entry, it emits this warning.

This warning is cosmetic and does not affect access to the requested devices. The device nodes remain accessible from the container because access is granted by the group ID and does not require a corresponding group name.

Mitigations and Workarounds

If you want to suppress the warning, you can disable the injection of additional group IDs for device nodes by enabling the `no-additional-gids-for-device-nodes` feature flag. Note that disabling this feature can prevent non-root users in a container from accessing device nodes that rely on group ownership for access, such as those used for video and rendering.

The feature is applied differently depending on how devices are injected into the container:

- When the NVIDIA Container Runtime injects devices (for example, when setting `--runtime=nvidia`), enable the feature flag in `/etc/nvidia-container-runtime/config.toml`:

```
[features]
no-additional-gids-for-device-nodes = true
```

- When generating a CDI specification with `nvidia-ctl cdi generate`, pass the feature flag so that the generated specification does not include the additional group IDs:

```
$ sudo nvidia-ctl cdi generate --feature-flag no-additional-gids-for-device-nodes
```

- When CDI specifications are generated by the `nvidia-cdi-refresh` systemd service, add the feature flag to `/etc/nvidia-container-toolkit/nvidia-cdi-refresh.env`:

```
NVIDIA_CTK_CDI_GENERATE_FEATURE_FLAGS=no-additional-gids-for-device-nodes
```



[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2020-2026, NVIDIA Corporation.